ALGORITHM

Start

Input URL

Check for suspicious keywords

Check for numbers

Check URL length

Check HTTPS

If any condition is true → Mark as Fraud

Else → Mark as Safe

Display result (Block/Allow)

End

PROJECT METHODOLOGY

a) Keyword Detection

Checks for suspicious words like:

login, verify, secure, update, free, bonus

Logic:

If any keyword is found → mark as suspicious

Reason:

Phishing URLs often use urgency or reward-based words

b) Number Detection

Checks if the URL contains digits

Reason:

Fraudulent domains often include random numbers to mimic legit sites

c) Length Check

Flags URLs longer than 40 characters

Reason:

Attackers use long URLs to hide malicious parts

d) HTTPS Verification

Checks if URL starts with https://

Reason:

Secure websites use HTTPS encryption

3. Decision Engine

A boolean flag suspicious is used

If any rule fails:

→ URL is marked fraudulent

Otherwise:

→ URL is marked safe

4. Output Generation

Displays warnings for each triggered rule

Final result:

Fraud Link → Access Blocked

Safe Link → Access Granted

PROJECT REQUIRMENTS

Functional Requirements

Accept a URL as input

Detect:

Suspicious keywords

Presence of numbers

URL length

HTTPS usage

Display warnings

Provide final classification (Safe / Fraud)

Non-Functional Requirements

Simple and fast execution

Easy to understand logic

No external dependencies

Runs on any standard C++ compiler

⚙️ System Requirements

C++ compiler (e.g., GCC)

Standard libraries:

<iostream>

<string>

<cctype>

PROJECT CODE

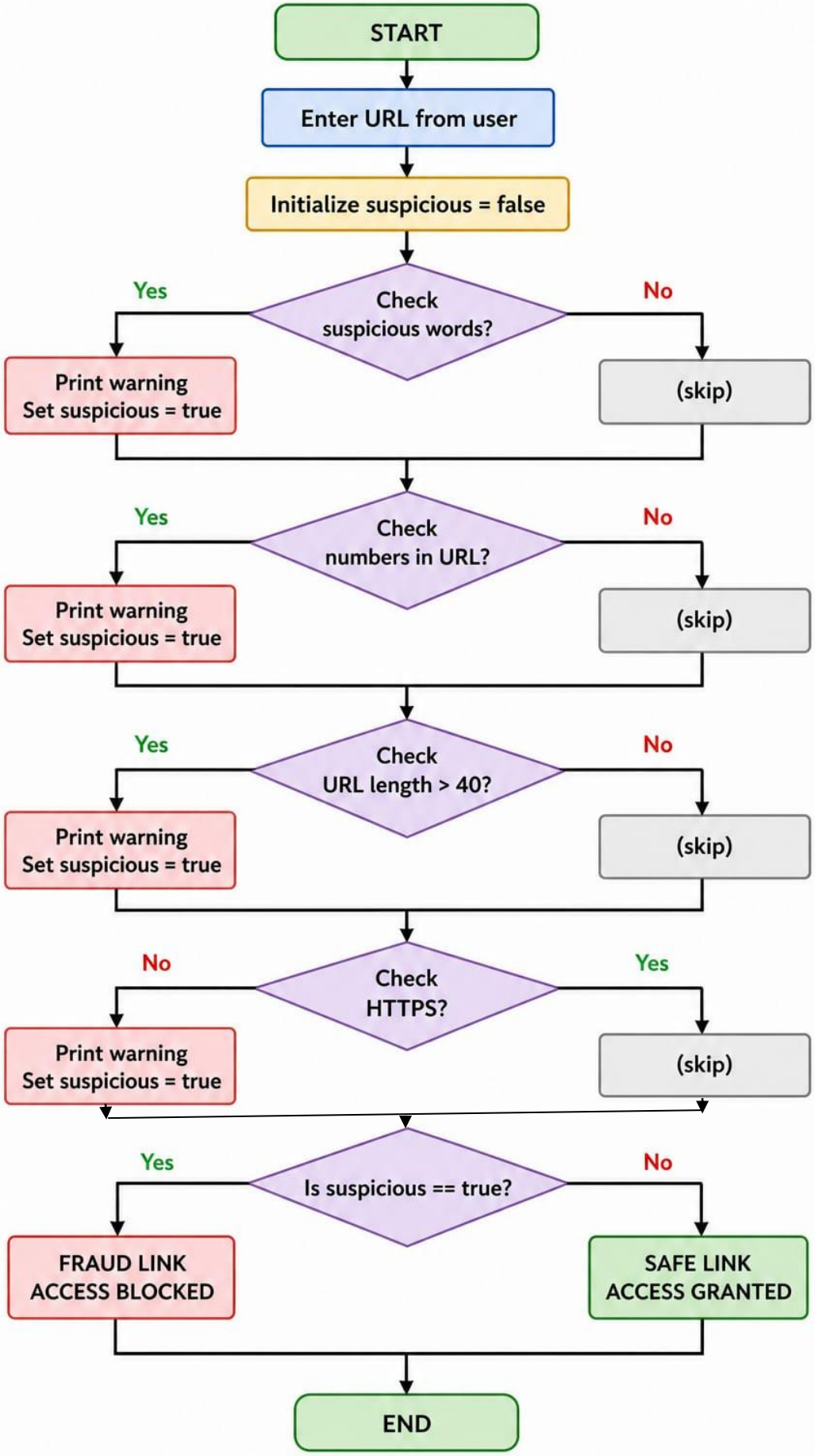
```
include <iostream>

#include <string>

using namespace std;


// Check for suspicious keywords

bool hasSuspiciousWords(string url) {
```

```
    string keywords[] = {"login", "verify", "secure", "update", "free", "bonus"};

    for (string word : keywords) {
        if (url.find(word) != string::npos) {
            return true;
        }
    }
    return false;
}

// Check if URL contains numbers
bool hasNumbers(string url) {
    for (char c : url) {
        if (isdigit(c)) return true;
    }
    return false;
}

// Check if URL is too long
bool isTooLong(string url) {
    return url.length() > 40;
}

// Check HTTPS
bool isSecure(string url) {
```

```
    return url.find("https://") == 0;
}

int main() {
    string url;

    cout << "Enter URL: ";
    cin >> url;

    bool suspicious = false;

    // Apply rules
    if (hasSuspiciousWords(url)) {
        cout << "⚠Contains suspicious keywords\n";
        suspicious = true;
    }

    if (hasNumbers(url)) {
        cout << "⚠Contains numbers\n";
        suspicious = true;
    }

    if (isTooLong(url)) {
        cout << "⚠URL is too long\n";
        suspicious = true;
```

```
    }

    if (!isSecure(url)) {
        cout << "⚠Not using HTTPS\n";
        suspicious = true;
    }

    // Final decision
    cout << "\n----------------------------\n";

    if (suspicious) {
        cout << "❌FRAUD LINK DETECTED!\n";
        cout << "   ACCESS BLOCKED\n";
        cout << "This link is unsafe. You cannot proceed.\n";
    } else {
        cout << "✅SAFE LINK\n";
        cout << "   ACCESS GRANTED\n";
        cout << "You can safely proceed to the website.\n";
    }

    cout << "----------------------------\n";

    return 0;
}
```

PROJECT FLOWCHART :


START
Enter URL from user
Initialize suspicious = false
Check suspicious words?
Yes
No
Print warning
Set suspicious = true
(skip)
Check numbers in URL?
Yes
No
Print warning
Set suspicious = true
(skip)
Check URL length > 40?
Yes
No
Print warning
Set suspicious = true
(skip)
Check HTTPS?
No
Yes
Print warning
Set suspicious = true
(skip)
Is suspicious == true?
Yes
No
FRAUD LINK
ACCESS BLOCKED
SAFE LINK
ACCESS GRANTED
END